

WEEK - 3 MODULE

MERGE SORT ALGORITHM

Ques-1

Input :-

6
450 1200 800 650 300 1500

Output

300 450 650 800 1200 1500

Logic

- ① Read the number of package values N .
- ② Store all ' N ' packages values in an array.
- ③ Call the merge sort function by passing the starting index (0) and the ending index ($N-1$)
- ④ In the merge sort function, check if the starting index is smaller than the ending index. If not, return because a single element is already sorted.
- ⑤ Find the middle index of the current array using:
$$\text{mid} = \text{left} + (\text{right} - \text{left}) / 2.$$
- ⑥ Recursively apply merge sort on the left half of the array.
- ⑦ Recursively apply merge sort on the right half of the array.

- ⑧ Create two temporary arrays to store the left and right sorted halves.
- ⑨ Compare the elements of both temporary arrays one by one.
- ⑩ Place the smaller element into the original array and move to the next element of that temporary array.
- ⑪ Continue comparing until all elements from one temporary array are used.
- ⑫ Copy the remaining elements (if any) from the right temporary array into the original array.
- ⑬ Copy the remaining element from the right temporary array into the original array.
- ⑭ Repeat the divide and merge process until the complete array becomes sorted.
- ⑮ Finally, print the sorted package values in ascending order.

Code

```
#include <iostream>
#include <vector>
using namespace std;
void merge (vector<int> &arr, int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;
    vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];
    int i = 0, j = 0, k = left;
    while (i < n1 & j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
}
```



```

while ( i < n1 ) {
    arr[k] = L[i];
    i++;
    k++;
}

```

```

while ( j < n2 ) {
    arr[k] = R[j];
    j++;
    k++;
}

```

```

}
mergesort (vector<int> &arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergesort (arr, left, mid);
        mergesort (arr, mid + 1, right);
        merge (arr, left, mid, right);
    }
}

```

```

}
int main () {
    int N;
    cin >> N;
    vector<int> arr(N);
    for (int i = 0; i < N; i++) {
        cin >> arr[i];
    }
}

```

```
mergecost(arr, 0, N-1);  
for (int i = 0; i < N; i++) {  
    cout << arr[i];  
    if (i != N-1)  
        cout << " ";  
}  
return 0;
```

```
}
```

```
PS C:\Users\pragy\OneDrive\Desktop\week module3> cd "c:\Users\pragy\OneDrive\Desktop\week module3"
; if ($?) { g++ merge1.cpp -o merge1 } ; if ($?) { .\merge1 }
6
450 1200 800 650 300 1500
300 450 650 800 1200 1500
PS C:\Users\pragy\OneDrive\Desktop\week module3>
```

Ques-2 - MERGE SORT ALGORITHM

Input

7

12 25 18 30 15 22 40

Output

12 15 18 22 25 30 40

Median : 22

Orders above median : 3

Difference : 28

Logic

- ① Start the program and take the input value N .
- ② Create an array to store all the processing time.
- ③ Read all N processing times from the user.
- ④ Apply merge sort on the array:
 - divide the array into two equal halves recursively.
 - keep dividing until each sub-array contains only one element.
 - merge the sub-array back in sorted order by comparing elements.
 - Continue merging until the complete array become sorted.
- ⑤ Display the sorted processing times in ascending order.
- ⑥ Find the median processing time:

→ If N is odd, the median is the middle element at index $N/2$.

⑦ Count the orders greater than the median:

→ Traverse the sorted array.

→ Compare each element with the median.

→ Increase the counter whenever an element is greater than the median.

⑧ Find the difference between the fastest and slowest processing time.

⑨ Print the final output:

→ sorted processing times

→ median processing times.

→ No. of orders above the median.

→ difference between maximum and minimum processing times.

⑩ End the program.

Code :

```
#include <iostream>
#include <vector>
using namespace std;
void merge (vector<int> &arr, int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;
    vector<int> L(n1), R(n2);
    for (int i=0; i<n1; i++)
        L[i] = arr[left + i];
    for (int j=0; j<n2, j++)
        R[j] = arr[mid + 1 + j];
    int i=0, j=0, k=left;
    while (i<n1 & j<n2)
    {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }
    while (i<n1)
        arr[k++] = L[i++];
    while (j<n2)
        arr[k++] = R[j++];
}
```

```
void mergesort (vector<int> &arr, int left, int right)
```

```
{
```

```
    if (left < right)
```

```
    {
```

```
        int mid = left + (right - left) / 2;
```

```
        mergesort (arr, left, mid);
```

```
        mergesort (arr, mid + 1, right);
```

```
        merge (arr, left, mid, right);
```

```
    }
```

```
}
```

```
int main () {
```

```
    int N;
```

```
    cin >> N;
```

```
    vector<int> arr (N);
```

```
    for (int i=0; i<N; i++)
```

```
        cin >> arr[i];
```

```
    mergesort (arr, 0, N-1);
```

```
    for (int i=0; i<N; i++)
```

```
        cout << arr[i] << " ";
```

```
    cout << endl;
```

```
    int median = arr [N/2];
```

```
    cout << " median :" << median << endl;
```

```
    int count = 0;
```

```
    for (int i=0; i<N; i++)
```

```
    {
```

```
if (arr[i] > median)
```

```
count ++;
```

```
}
```

```
cout << "Orders Above Median:" << count << endl;
```

```
int difference = arr[N-1] - arr[0];
```

```
cout << "Difference:" << difference << endl;
```

```
return 0;
```

```
}
```

```
PS C:\Users\pragy\OneDrive\Desktop\week module3> cd "c:\Users\pragy\OneDrive\Desktop\week module3\" ; if ($?) { g++ merge2.cpp -o merge2 } ; if ($?) { .\merge2 }
7
12 25 18 30 15 22 40
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28
PS C:\Users\pragy\OneDrive\Desktop\week module3> █
```


QUICK SORT

Ques-1

Input:

6

120 45 78 200 60 95

Output

45 60 78 95 120 200

Logic

- ① Start the program and input the value of N .
- ② Create an array to store all the response time.
- ③ Read all N response time from the user.
- ④ Apply quick sort to sort the array in ascending order:
 - select the last element as the pivot.
 - Compare each element with the pivot.
 - Place all element smaller than or equal to the pivot on its left side.
 - Place all elements greater than the pivot on its right side.
 - Put the pivot at its correct sorted position.
- ⑤ Recursively apply quick sort;
 - sort the left part of the pivot.

→ sort the right part of the pivot.

→ continue until each sub-array contains only one or zero elements.

⑥ After the complete sorting process, the array will be arranged from the smallest response time to the largest response time.

⑦ Traverse the sorted array and print each response time in ascending order.

⑧ End the program.

Code

```
#include <iostream>
#include <vector>
using namespace std;
int partition (vector <int> & arr, int low, int high)
{
    int pivot = arr [high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        i++;
        swap (arr [i], arr [j]);
    }
    if (arr [j] <= pivot)
    {
        i++;
    }
}
```

```

        swap(arr[i], arr[j]);
    }

    swap(arr[i+1], arr[high]);
    return i+1;
}

void quicksort (vector<int> &arr, int low, int high)
{
    if (low < high)
    {
        int pivotIndex = partition (arr, low, high);
        quicksort (arr, low, pivotIndex-1);
        quicksort (arr, pivotIndex+1, high);
    }
}

int main ()
{
    int N;
    cin >> N;
    vector<int> arr (N);
    for (int i = 0; i < N; i++)
    {
        cin >> arr[i];
    }
    quicksort (arr, 0, N-1);
    for (int i = 0; i < N; i++)
    {

```



```
cout << arr[i] << " ";
```

```
y
```

```
return 0;
```

```
y.
```



```
> cd "c:\Users\pragy\OneDrive\Desktop\week module3\" ; if ($?) { g++ quick1.cpp -o quick1 } ; if ($?) { .\quick1 }  
6  
120 45 78 200 60 95  
45 60 78 95 120 200  
PS C:\Users\pragy\OneDrive\Desktop\week module3>
```

Ques-2

Input:-

8

500 1200 700 2000 900 1500 3000 2500

Output

3000 2500 2000 1500 1200 900 700 500

Top 5: 3000 2500 2000 1500 1200

Average of Top 5: 2040

Values above overall Average: 4.

Logic

1. Read the number of trade values N from the user.
2. Store all N trade values in an array or vector.
3. Apply quick sort to sort the array in descending order.
 - Select the last element as the pivot.
 - Move all values greater than or equal to the pivot to the left side.
 - Place the pivot at its correct position.
 - Recursively repeat the same process for the left and right parts until the array is completely sorted.
4. Print the sorted trade values from highest to lowest.
5. Find the Top 5 highest trade values.
 - Since the array is already sorted in descending order, the first five elements are the Top 5 values.

6. Display these Top 5 trade values.
7. Calculate the sum of the Top 5 values.
8. Find the average of the Top 5 values by dividing the sum by 5.
9. Display the average of Top 5 values.
10. Calculate the sum of all trade values in the array.
11. Find the overall average by dividing the total sum by N.
12. Traverse the sorted array and count how many trade values are greater than the overall average.
13. Display the count of values greater than the overall average.
14. End the program after printing:

→ Sorted trade values

→ Top 5 trade values.

→ Average of Top 5 values.

→ Count the values greater than the overall average.

Code

```
#include <iostream>
#include <vector>
using namespace std;
int partition (vector <long long> &arr, int low, int high)
{
    long long pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        if (arr[j] >= pivot)
        {
            i++;
            swap (arr[i], arr[j]);
        }
    }
    swap (arr[i+1], arr[high]);
    return i+1;
}

void quicksort (vector <long long> &arr, int low, int high)
{
    if (low < high)
    {
        int pivotIndex = partition (arr, low, high);
        quicksort (arr, low, pivotIndex - 1);
        quicksort (arr, pivotIndex + 1, high);
    }
}
```



```
int main ()
```

```
{
```

```
    int N;
```

```
    cin >> N;
```

```
    vector< long long > tradeValues (N);
```

```
    for (int i=0; i<N; i++)
```

```
    {
```

```
        cin >> tradeValues [i];
```

```
    }
```

```
    quickSort (tradeValues, 0, N-1);
```

```
    for (int i=0; i<N; i++)
```

```
    {
```

```
        cout << tradeValues [i] << " ";
```

```
    }
```

```
    cout << endl;
```

```
    cout << "Top 5: ";
```

```
    long long topSum = 0;
```

```
    for (int i=0; i<5; i++)
```

```
    {
```

```
        cout << tradeValues [i] << " ";
```

```
        topSum += tradeValues [i];
```

```
    }
```

```
    cout << endl;
```

```
    cout << "Average of Top 5: " << topSum / 5 << endl;
```

```
    long long totalSum = 0;
```

```
    for (int i=0; i<N; i++)
```

```
    {
```

```
if (tradeValues[i] > overallAverage)
```

```
{
```

```
    count ++;
```

```
}
```

```
}
```

```
cout << "Values Above Overall Average!" <<
```

```
count << endl;
```

```
return 0;
```

```
}
```

```
p\week module3\" ; if ($?) { g++ quick2.cpp -o quick2 } ; if ($?) { .\quick2 }  
8  
500 1200 700 2000 900 1500 3000 2500  
3000 2500 2000 1500 1200 900 700 500  
Top 5: 3000 2500 2000 1500 1200  
Average of Top 5: 2040  
Values Above Overall Average: 3  
PS C:\Users\pragy\OneDrive\Desktop\week module3> |
```

HEAP SORT

Ques - 1

Input :

6
850 1200 950 600 1500 1000

Output:

600 850 950 1000 1200 1500.

Logic

1. Read the number of player scores N from the user.
2. Store all N player scores in an array or vector.
3. Create a Max Heap from the given array.
 - Start from the last non-leaf node.
 - Compare the parent with its left and right children.
 - If any child is larger than the parent, swap them.
 - Repeat this process until the subtree satisfies the Heap property.
4. Once the maxheap is built, the largest score will be at the root.
5. Swap the first element (largest score) with the last unsorted element.
6. Reduce the heap size by one, as the last element is now in its correct sorted position.
7. Apply Heapify on the root to restore the MaxHeap property for the remaining elements.

8. Repeat step 5 to 7 until only one element remains in the heap.
9. After completing all iterations, the array will be sorted in ascending order.
10. Traverse the sorted array and print all player scores in increasing order.
11. End the program.

Code

```
#include <iostream>
#include <vector>
using namespace std;
void heapify (vector<int> &arr, int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n & arr[left] > arr[largest])
        largest = left;
    if (right < n & arr[right] > arr[largest])
        largest = right;
    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}
```

}

}

```

void heapsort (vector<int> &arr, int n) {
    for (int i = n/2 - 1; i >= 0; i--)
        heapify (arr, n, i);
    for (int i = n-1; i > 0; i--) {
        swap (arr[0], arr[i]);
        heapify (arr, i, 0);
    }
}

```

```

}
int main () {
    int n;
    cin >> n;
    vector<int> arr (n);
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    heapsort (arr, n);
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}

```

```
PS C:\Users\pragy\OneDrive\Desktop\week module3> cd "c:\Users\pragy\OneDrive\Desktop\week module3\" ; if ($?) { g++ heap1.cpp -o heap1 } ; if ($?) { .\heap1 }  
6  
850 1200 950 600 1500 1000  
600 850 950 1000 1200 1500  
PS C:\Users\pragy\OneDrive\Desktop\week module3> █
```

Ques-2 (Heap Sort)

Input

8

12 7 15 9 20 5 10 8

Output

5 7 9 10 12 15 20

Fastest : 5

Slowest : 20

Average : 10.75

Cases Faster Than Average : 5

Percentage : 62.50%

Logic

1. Read the number of response times N from the user.
2. Store all N response times in an array or vector.
3. Create a Max Heap from the given array.
 - Start from the last non-leaf node.
 - Compare the parent with its left and right child.
 - If any child is greater than the parent, swap them.
 - Repeat the process until the subtree satisfies the max heap property.
4. Apply Heapsort to sort the response time in ascending order.
 - Swap the root with the last element.

- Reduce the heap size by one.
- Heapify the remaining heap.
- Repeat until the entire array is sorted.

5. Display the sorted response times.

6. Since the array is sorted in ascending order.

- the first element is the fastest response time.
- the last element is the slowest response time.

7. Print the fastest and slowest response times.

8. Traverse the array and calculate the sum of all response times.

9. Find the average response time by dividing the total sum by N .

10. Display the average response time.

11. Traverse the array again and count how many response times are less than the average.

12. Display the number of cases handled faster than the average.

13. Calculate the percentage of cases faster than the average using:

$$\rightarrow \text{Percentage} = (\text{count} \times 100.0) / N$$

14. Display the percentage upto 2-decimal places.

→ No. of cases faster than avg.

→ Percentage of cases faster than average.

Code

```
#include <iostream>
#include <vector>
#include <iomanip>
using namespace std;
void heapify (vector<int> &arr, int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;
    if (largest != i)
    {
        swap (arr[i], arr[largest]);
        heapify (arr, n, largest);
    }
}

void heapsort (vector<int> &arr, int n)
{
    for (int i = n/2 - 1; i >= 0; i--)
    {
        heapify (arr, n, i);
    }
}
```

```

for (int i = n - 1; i > 0; i--)
{
    swap (arr[0], arr[i]);
    heapify (arr, i, 0);
}

```

```

}
int main ()
{

```

```

    int N ;

```

```

    cin >> N;

```

```

    vector <int> responseTime (N);

```

```

    for (int i = 0; i < N; i++)
    {

```

```

        cin >> responseTime[i];
    }

```

```

}

```

```

    heapSort (responseTime, N);

```

```

    for (int i = 0; i < N; i++)
    {

```

```

        cout << responseTime[i] << " ";
    }

```

```

}

```

```

    cout << endl;

```

```

    cout << "fastest:" << responseTime[0] << endl;

```

```

    cout << "slowest:" << responseTime[N-1] << endl;

```

```

    int sum = 0;

```

```

    for (int i = 0; i < N; i++)
    {

```

```

    {

```

```

        sum += responseTime[i];
    }

```

```

}

```



```

double average = (double)sum / N;
cout << fixed << setprecision (2);
cout << "Average:" << average << endl;
int count = 0;
for (int i = 0; i < N; i++)
{

```

```

    if (responseTime[i] < average)
    {
        count++;
    }
}

```

```

cout << "Cases Faster Than Average : " << count <<
    endl;

```

```

double percentage = (count * 100.0) / N;

```

```

cout << "Percentage : " << percentage << "%" << endl;
return 0;
}

```



```
> cd "c:\Users\pragy\OneDrive\Desktop\week module3\" ; if ($?) { g++ heap2.cpp -o heap2 } ; if ($?) { .\heap2 }  
8  
12 7 15 9 20 5 10 8  
5 7 8 9 10 12 15 20  
Fastest: 5  
Slowest: 20  
Average: 10.75  
Cases Faster Than Average: 5  
Percentage: 62.50%  
PS C:\Users\pragy\OneDrive\Desktop\week module3> |
```